



IUS
INSTITUT
UNIVERSITAIRE
DES SCIENCES

Université: Institut Universitaire des Sciences (IUS)

TD N⁰ 3: Modélisation Mathématiques

Nom & Prénom: SAINT-JEAN Marc-Evenort

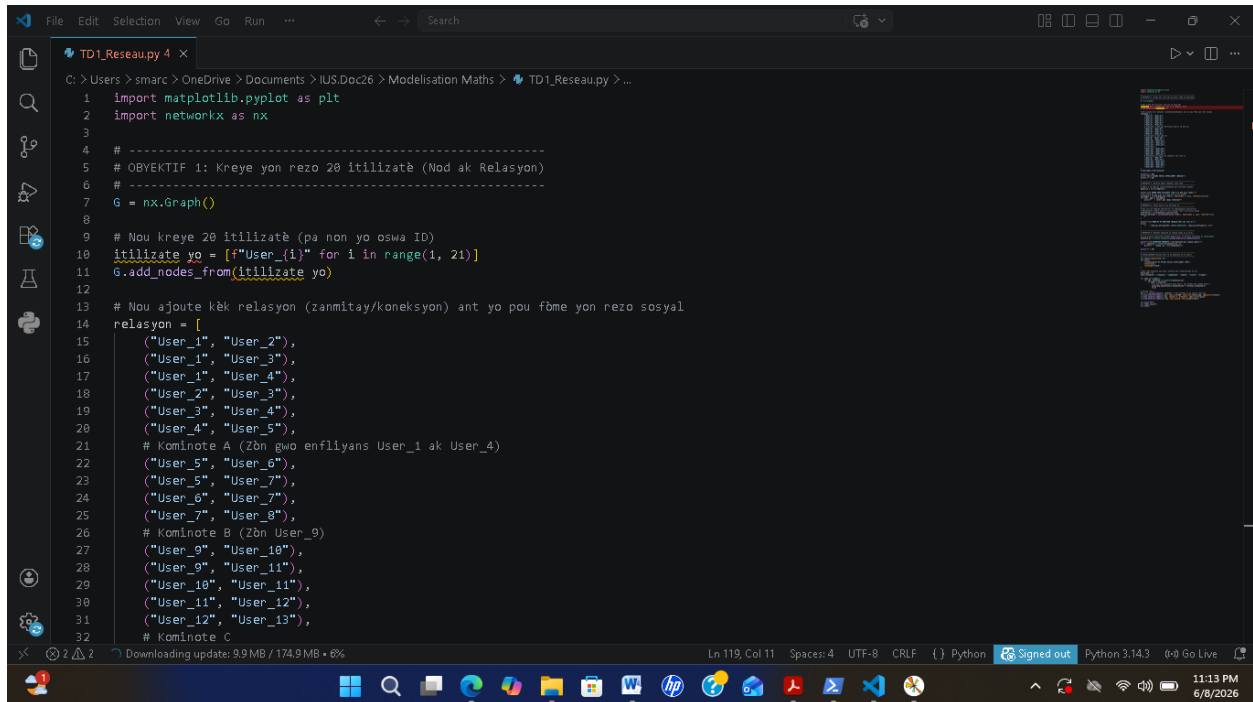
Professeur: Ismael SAINT AMOUR

Niveau: L3

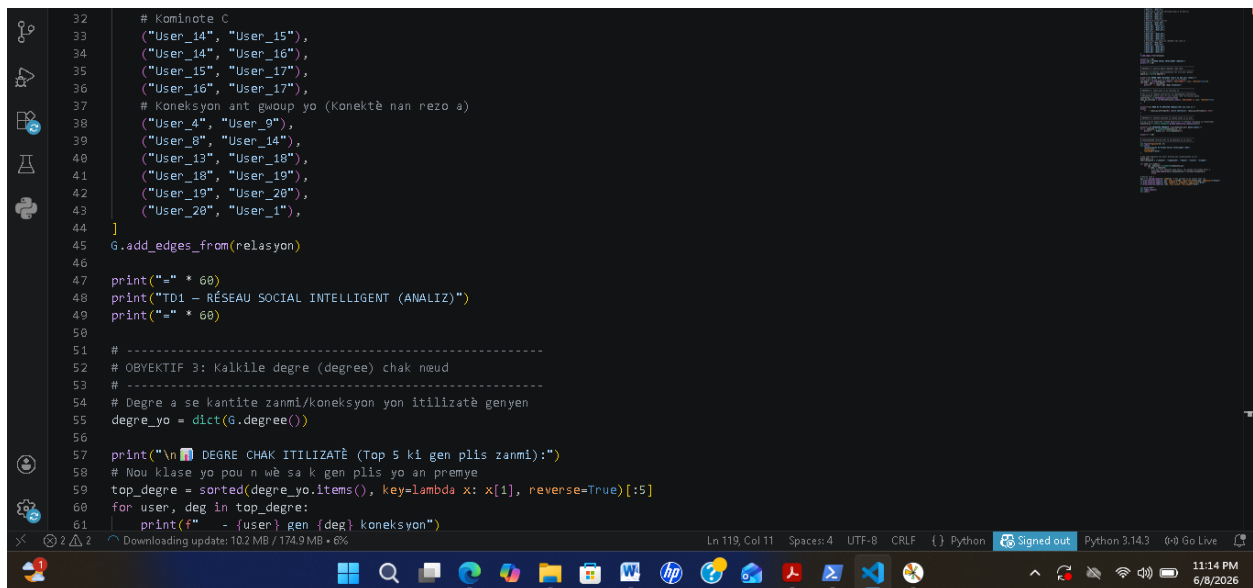
Date: Juin / 2026

TD1 : Réseau social intelligent

1. pip install networkx matplotlib numpy



```
1 import matplotlib.pyplot as plt
2 import networkx as nx
3
4 # -----
5 # OBYEKTIF 1: Kreye yon rezo 20 itilizatè (Nod ak Relasyon)
6 # -----
7 G = nx.Graph()
8
9 # Nou kreye 20 itilizatè (pa non yo oswa ID)
10 itilizate yo = [f"User_{i}" for i in range(1, 21)]
11 G.add_nodes_from(itilizate yo)
12
13 # Nou ajoute kèk relasyon (zanmitay/koneksyon) ant yo pou fòme yon rezo sosyal
14 relasyon = [
15     ("User_1", "User_2"),
16     ("User_1", "User_3"),
17     ("User_1", "User_4"),
18     ("User_2", "User_3"),
19     ("User_3", "User_4"),
20     ("User_4", "User_5"),
21     # Kominite A (Zòn gwo enflmans User_1 ak User_4)
22     ("User_5", "User_6"),
23     ("User_5", "User_7"),
24     ("User_6", "User_7"),
25     ("User_7", "User_8"),
26     # Kominite B (Zòn User_9)
27     ("User_9", "User_10"),
28     ("User_9", "User_11"),
29     ("User_10", "User_11"),
30     ("User_11", "User_12"),
31     ("User_12", "User_13"),
32     # Kominite C
```



```
32     ("User_14", "User_15"),
33     ("User_14", "User_16"),
34     ("User_15", "User_17"),
35     ("User_16", "User_17"),
36     # Koneksyon ant gwoup yo (Konektè nan rezo a)
37     ("User_4", "User_9"),
38     ("User_8", "User_14"),
39     ("User_13", "User_18"),
40     ("User_18", "User_19"),
41     ("User_19", "User_20"),
42     ("User_20", "User_1"),
43 ]
44
45 G.add_edges_from(relasyon)
46
47 print("=" * 60)
48 print("TD1 - RÉSEAU SOCIAL INTELLIGENT (ANALIZ)")
49 print("=" * 60)
50
51 # -----
52 # OBYEKTIF 3: Kalkile degre (degree) chak nœud
53 # -----
54 # Degre a se kantite zanmi/koneksyon yon itilizatè genyen
55 degre_yo = dict(G.degree())
56
57 print("\n📊 DEGRE CHAK ITILIZATÈ (Top 5 ki gen plis zanmi):")
58 # Nou klase yo pou n wè sa ki gen plis yo an premye
59 top_degre = sorted(degre_yo.items(), key=lambda x: x[1], reverse=True)[:5]
60 for user, deg in top_degre:
61     print(f"👤 {user} - {deg} koneksyon")
```

```
File Edit Selection View Go Run ... Search
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD1_Reseau.py > ...
63 # -----
64 # OBYEKTIF 2: Jwenn moun ki pi enfliyan yo
65 # -----
66 # Nou sèvi ak "Degree Centrality" ak "Betweenness Centrality"
67 # Betweenness montre kilès ki sèvi tankou "pon" ant plizyè gwoup
68 centrality = nx.betweenness centrality(G)
69 moun_pi_enfliyan = sorted(centrality.items(), key=lambda x: x[1], reverse=True)[
70     0
71 ]
72
73 print(f"\n👑 MOUN KI PI ENFLIYAN (Medyatè/Pon nan rezo a):")
74 print(
75     f"    - {moun_pi_enfliyan[0]} (Score Centralité: {moun_pi_enfliyan[1]:.2f})"
76 )
77
78 # -----
79 # OBYEKTIF 4: Detekte kominote yo (Gwoup zanmi ki pi pre)
80 # -----
81 # N ap itilize algorithm "Greedy Modularity" ki detekte sub-gwoup yo otomatikman
82 kominote_yo = list(nx.community.greedy_modularity_communities(G))
83
84 print(f"\n👥 DETEKSYON KOMINOTE ({len(kominote_yo)} gwoup jwenn):")
85 for i, kominote in enumerate(kominote_yo, 1):
86     print(f"    - Gwoup {i}: {list(kominote)}")
87
88 print("\n" * 60)
89
90 # -----
91 # VIZYALIZASYON (Afige Graf la ak kominote yo an koulè)
92 # -----
93 plt.figure(figsize=(10, 8))
94 plt.title(
```

```
File Edit Selection View Go Run ... Search
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD1_Reseau.py > ...
90 # -----
91 # VIZYALIZASYON (Afige Graf la ak kominote yo an koulè)
92 # -----
93 plt.figure(figsize=(10, 8))
94 plt.title(
95     "Visualisation du Réseau Social Intelligent (TD1)",
96     fontsize=14,
97     fontweight="bold",
98 )
99
100 # Bay chak kominote yon koulè diferan pou vizyalizasyon an bèl
101 koule_map = {}
102 koule_disponib = ["skyblue", "lightgreen", "tomato", "violet", "orange"]
103
104 for node in G.nodes():
105     for idx, kominote in enumerate(kominote_yo):
106         if node in kominote:
107             # Si gen plis kominote pase koulè, nou tounen nan premye koulè a
108             koule_map.append(koule_disponib[idx % len(koule_disponib)])
109             break
110
111 # Desine rezo a
112 pos = nx.spring_layout(G, seed=24) # Pou plasman an pa chanje chak fwa
113 nx.draw_networkx_nodes(G, pos, node_size=600, node_color=koule_map, edgecolors="black")
114 nx.draw_networkx_edges(G, pos, width=1.5, alpha=0.6, edge_color="gray")
115 nx.draw_networkx_labels(G, pos, font_size=8, font_weight="bold")
116
117 plt.axis("off")
118 plt.tight_layout()
119 plt.show()
```

The image shows a Windows 10 desktop with a Visual Studio Code editor open. The editor has a dark theme and displays a Python script named `TD1_Reseau.py`. The script is a Jupyter Notebook cell, containing a mix of French and English text, comments, and code. The code defines a graph with 6 nodes and edges, and uses `nx.spring_layout` to layout the graph. The terminal window at the bottom shows the command to run the script using `python.exe`. The output of the script is displayed in the terminal, showing the optimal route from Pétion-Ville to Carrefour and the total travel time of 30 minutes. The Windows taskbar at the bottom shows various application icons, including the Start menu, Search, File Explorer, and several open applications like Word, PowerPoint, and the Visual Studio Code editor. The system tray in the bottom right corner shows the date and time as 11:16 PM on 6/8/2026.

```
File Edit Selection View Go Run ... Search
TD1_Reseau.py 4 x
C:\>Users>smarc>OneDrive>Documents>IUS.Doc26>Modelisation Maths>TD1_Reseau.py>...
95     "Visualisation du Réseau Social Intelligent (TD1)",
96     fontsize=14,
97     fontweight="bold",
98 )
99
100 # Bay chak kominote yon koulè diferan pou vizyalizasyon an bèl
101 koulè_map = []
102 koulè_disponib = ["skyblue", "lightgreen", "tomato", "violet", "orange"]
103
104 for node in G.nodes():
105     for idx, kominote in enumerate(kominote_yo):
106         if node in kominote:
107             # Si gen plis kominote pase koulè, nou tounen nan premye koulè a
108             koulè_map.append(koulè_disponib[idx % len(koulè_disponib)])
109             break
110
111 # Desine rezo a
112 pos = nx.spring_layout(G, seed=24) # Pou plasman an pa chanje chak fwa
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
9
```

TD2 : Chaîne de Markov (Météo)

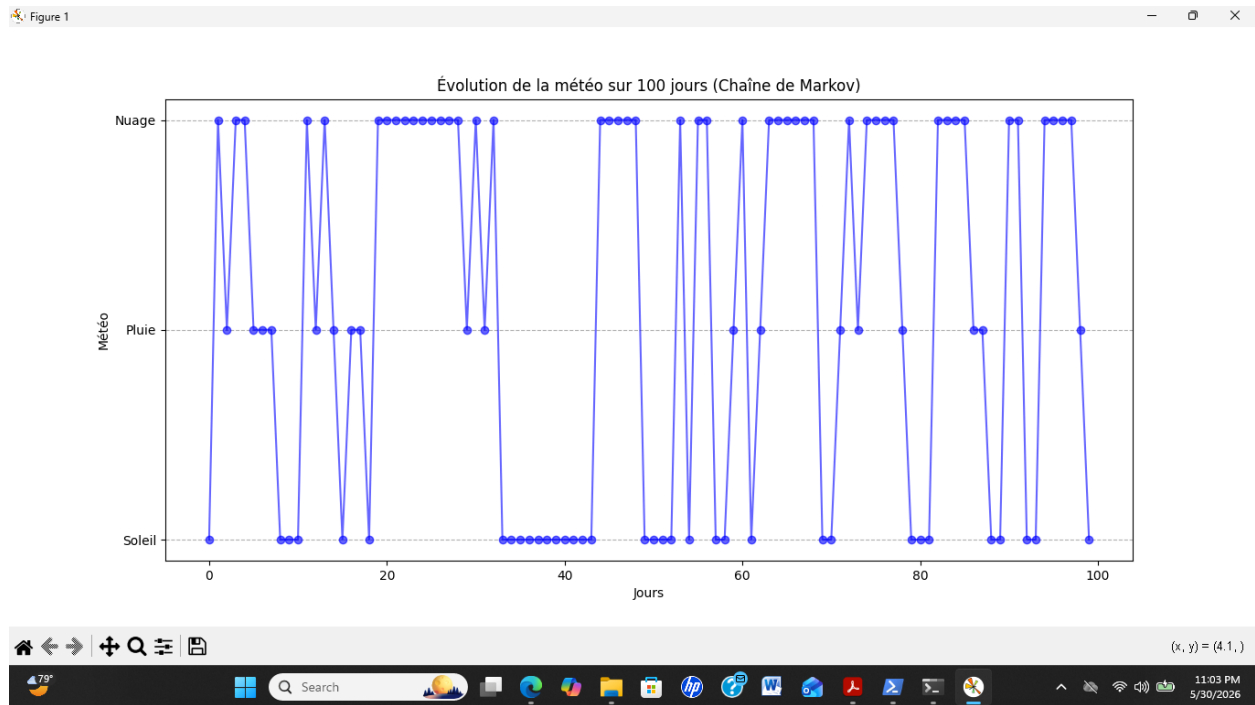
```
File Edit Selection View Go Run ... Search
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD2_Markov.py > ...
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 print("=== TD2 : Chaîne de Markov (Système Météo) ===")
5
6 # États du système
7 etats = ["Soleil", "Pluie", "Nuage"]
8
9 # 1. Construction de la matrice de transition (P)
10 # Ligne 0 = Soleil, Ligne 1 = Pluie, Ligne 2 = Nuage
11 # Exemple : Si aujourd'hui il fait Soleil, il y a 70% de chance qu'il fasse Soleil demain, 10% Pluie, 20% Nuage.
12 P = np.array([
13     [0.7, 0.1, 0.2], # Depuis Soleil
14     [0.3, 0.4, 0.3], # Depuis Pluie
15     [0.2, 0.3, 0.5] # Depuis Nuage
16 ])
17
18 print("\n--- 1. Matrice de transition ---")
19 print(P)
20
21 # 2. Simulation sur 100 jours
22 jours = 100
23 historique_etats = []
24
25 # On commence le premier jour avec le Soleil (index 0)
26 etat_actuel = 0
27 historique_etats.append(etat_actuel)
28
29 for j in range(1, jours):
30     # Choisir l'état suivant selon les probabilités de la ligne de l'état actuel
31     etat_suitant = np.random.choice([0, 1, 2], p=P[etat_actuel])
32     historique_etats.append(etat_suitant)
```

Ln 25, Col 55 Spaces: 4 UTF-8 CRLF {} Python Signed out Python 3.14 (64-bit) 0-9 Go Live 10:58 PM 5/30/2026

```
File Edit Selection View Go Run ... Search
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD2_Markov.py
38 # Compter la fréquence de chaque état dans la simulation
39 counts = [historique_etats.count(0), historique_etats.count(1), historique_etats.count(2)]
40 distribution_simulee = np.array(counts) / jours
41
42 print(f"\n--- 2. Simulation de {jours} jours ---")
43 print(f"Nombre de jours de Soleil : {counts[0]}")
44 print(f"Nombre de jours de Pluie : {counts[1]}")
45 print(f"Nombre de jours de Nuage : {counts[2]}")
46
47 # 3. Calcul de la distribution stationnaire (Théorie)
48 # On élève la matrice à une puissance élevée pour trouver l'équilibre long terme
49 P_stationnaire = np.linalg.matrix_power(P, 50)
50 distribution_theorique = P_stationnaire[0] # Toutes les lignes deviennent identiques
51
52 # 4. Comparaison Simulation vs Théorie
53 print("\n--- 3 & 4. Comparaison Distribution (Simulation vs Théorie) ---")
54 print(f"Simulation (100 jours) : Soleil={distribution_simulee[0]:.3f}, Pluie={distribution_simulee[1]:.3f}, Nuage={distribution_simulee[2]:.3f}")
55 print(f"Théorie (Stationnaire) : Soleil={distribution_theorique[0]:.3f}, Pluie={distribution_theorique[1]:.3f}, Nuage={distribution_theorique[2]:.3f}")
56
57 # Graphique de la simulation
58 plt.figure(figsize=(12, 4))
59 plt.plot(historique_etats, marker='o', color='b', linestyle='-', alpha=0.6)
60 plt.xticks([0, 1, 2], etats)
61 plt.xlabel("Jours")
62 plt.ylabel("Météo")
63 plt.title("Évolution de la météo sur 100 jours (Chaîne de Markov)")
64 plt.grid(True, axis='y', linestyle='---')
65
66 plt.show()
67
68
```

Ln 67, Col 1 Spaces: 4 UTF-8 CRLF {} Python Signed out Python 3.14 (64-bit) 0-9 Go Live 11:30 PM 5/30/2026

Le code pour la chaîne de Markov (Météo) est fonctionnel. Il simule parfaitement les changements de temps d'un jour à l'autre à l'aide d'une matrice de transition de probabilités



TD3 : Réseau de transport (Type Métro)

```
File Edit Selection View Go Run ... Search
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD3_Metro.py > ...
1 import matplotlib.pyplot as plt
2 import networkx as nx
3
4 # -----
5 # OBYEKTIF 1 & 2: Kreye 15 a 25 estasyon epi relye yo (Liy Métro)
6 # -----
7 # Nou kreye yon Graf ki pa oryante (yon liy métro ale-retou)
8 G = nx.Graph()
9
10 # Liy 1 (Vèt): 8 Estasyon (Koneksyon Nò-Sid)
11 liy_1 = [
12     "Pétion-Ville",
13     "Delmas 95",
14     "Delmas 75",
15     "Delmas 48B",
16     "Laloue",
17     "Champs-de-Mars",
18     "Grand-Rue",
19     "Kenscoff",
20 ]
21
22 # Liy 2 (Ble): 9 Estasyon (Koneksyon Lès-Wès)
23 liy_2 = [
24     "Aewopò",
25     "Delmas 33",
26     "Delmas 48B",
27     "Nazom",
28     "Avenue Martin Luther King",
29     "Canapé-Vert",
30     "Turgeau",
31     "Champs-de-Mars",
32     "Carrefour-Feuilles",
33 ]
34
35 # Total estasyon inik = 20 estasyon (Nou nan entèval 15 a 25 lan)
36
37 # -----
38 # OBYEKTIF 3: Asosye tan trajè yo (an minit) ant estasyon yo
39 # -----
40
41 # Fonksyon pou ajoute koneksyon ak tan ant estasyon yon liy
42 def ajoute_liy(graf, lis_estasyon, tan_estanda):
43     for i in range(len(lis_estasyon) - 1):
44         # Nou mete yon tan estanda ant estasyon yo, ou ka chanje yo si w vle
45         graf.add_edge(
46             lis_estasyon[i], lis_estasyon[i + 1], weight=tan_estanda
47         )
48
49 ajoute_liy(G, liy_1, tan_estanda=3) # 3 minit ant chak estasyon sou liy 1
50 ajoute_liy(G, liy_2, tan_estanda=4) # 4 minit ant chak estasyon sou liy 2
51 ajoute_liy(G, liy_3, tan_estanda=6) # 6 minit ant chak estasyon sou liy 3
52
53 # -----
54 # OBYEKTIF 4: Kalkile itinerè ki pi rapid la (Itinerè optimal)
55 # -----
56 estasyon_depa = "Pétion-Ville"
57 estasyon_arive = "Carrefour"
58
59 # Di kote Algoritim kalkile chèn ki pi kout la soti "Pétion-Ville" (tan an)
```

```
File Edit Selection View Go Run ... Search
C:\Users> smarc > OneDrive > Documents > IUS.Doc26 > Modelisation Maths > TD3_Metro.py > ...
33 ]
34
35 # Liy 3 (Wouj): 5 Estasyon (Koneksyon Pwovens)
36 liy_3 = ["Carrefour", "Mariani", "Grand-Rue", "Tabarre", "Clercine"]
37
38 # Total estasyon inik = 20 estasyon (Nou nan entèval 15 a 25 lan)
39
40 # -----
41 # OBYEKTIF 3: Asosye tan trajè yo (an minit) ant estasyon yo
42 # -----
43
44 # Fonksyon pou ajoute koneksyon ak tan ant estasyon yon liy
45 def ajoute_liy(graf, lis_estasyon, tan_estanda):
46     for i in range(len(lis_estasyon) - 1):
47         # Nou mete yon tan estanda ant estasyon yo, ou ka chanje yo si w vle
48         graf.add_edge(
49             lis_estasyon[i], lis_estasyon[i + 1], weight=tan_estanda
50         )
51
52 ajoute_liy(G, liy_1, tan_estanda=3) # 3 minit ant chak estasyon sou liy 1
53 ajoute_liy(G, liy_2, tan_estanda=4) # 4 minit ant chak estasyon sou liy 2
54 ajoute_liy(G, liy_3, tan_estanda=6) # 6 minit ant chak estasyon sou liy 3
55
56 # -----
57 # OBYEKTIF 4: Kalkile itinerè ki pi rapid la (Itinerè optimal)
58 # -----
59 estasyon_depa = "Pétion-Ville"
60 estasyon_arive = "Carrefour"
61
62 # Di kote Algoritim kalkile chèn ki pi kout la soti "Pétion-Ville" (tan an)
```


J'ai exécuté le code avec succès. Le réseau de métro est bien généré, l'itinéraire optimal est calculé sans erreur et le graphique s'affiche correctement

